TP2 - File Integrity Checker and IOC Matcher in Rust

Module 7.1 - Programming with Rust

Student: Abderahman

Date: June 15, 2026

GitHub: https://github.com/x0abderahman/tp2_integrity_checker

Defensive Security - File Integrity Verification Tool

1. Objective

This project implements a command-line File Integrity Checker and IOC (Indicator of Compromise) Matcher written in Rust. The tool scans a file or directory, calculates SHA-256 cryptographic hashes for each regular file, compares them against a provided list of known malicious hashes (IOCs), and produces a structured CSV report.

File integrity checking is a fundamental defensive security technique. It allows security analysts to verify that files have not been tampered with by comparing their current cryptographic hash against a known good or known bad baseline. An IOC (Indicator of Compromise) is a piece of forensic evidence - such as a file hash, IP address, or domain name - that indicates a potential security breach.

The tool is designed with secure programming practices: it avoids unsafe code, handles errors gracefully without panicking, and splits functionality into well-defined modules.

2. Environment

The development environment uses a Docker Compose setup with Debian Bookworm:

```
mkdir -p workspace
docker compose up -d --build
docker compose exec rustlab bash
cd /workspace/tp2_integrity_checker
```

Rust toolchain versions:

```
rustc 1.95.0 (59807616e 2026-04-14)
cargo 1.95.0 (f2d3ce0bd 2026-03-21)
rustfmt 1.9.0-stable
clippy 0.1.95
```

Project path inside the container: /workspace/tp2_integrity_checker

Figure 1: Rust environment and toolchain versions

3. Implementation

3.1 Hashing Module (hashing.rs)

The hashing module implements the SHA-256 hash calculation using the sha2 crate. Files are read in 8KB buffers to handle large files efficiently without loading the entire content into memory. The hash is returned as a lowercase hexadecimal string.

```
pub fn hash_file_sha256(path: &Path) -> Result<String, io::Error> {
    let mut file = File::open(path)?;
    let mut hasher = Sha256::new();
    let mut buffer = [0u8; 8192];
    loop {
        let bytes_read = file.read(&mut buffer)?;
        if bytes_read == 0 { break; }
        hasher.update(&buffer[..bytes_read]);
    }
    Ok(format!("{:x}", hasher.finalize()))
}
```

3.2 IOC Parsing Module (ioc.rs)

The IOC module parses a CSV-like file containing hashes and labels. It handles: empty lines (ignored), comment lines starting with # (ignored), invalid SHA-256 strings (counted but skipped), and valid entries (parsed into IocEntry structs). The is_valid_sha256 function validates that strings are exactly 64 hexadecimal characters.

3.3 Scanner Module (scanner.rs)

The scanner module walks through a target path. If it is a single file, it scans it directly. If it is a directory, it iterates over all regular files. Each file's hash is calculated and compared against the IOC list. Results are stored as ScanResult structs with Clean, Match, or Error status.

3.4 Report Module (report.rs)

The report module generates a CSV file with columns: path, sha256, status, label. It uses the csv crate for proper encoding and handles the report directory creation.

3.5 CLI Handling (main.rs)

Command-line arguments are parsed manually for --target, --ioc, and --report flags. Missing arguments or errors produce a clear usage message. The main function delegates to a run() function that returns Result, avoiding unwrap() in the execution path.

```
Usage:
  tp2_integrity_checker --target <FILE_OR_DIR> --ioc <IOC_FILE> --report <REPORT>
```

4. Execution Evidence

The following screenshots demonstrate successful execution of the tool:

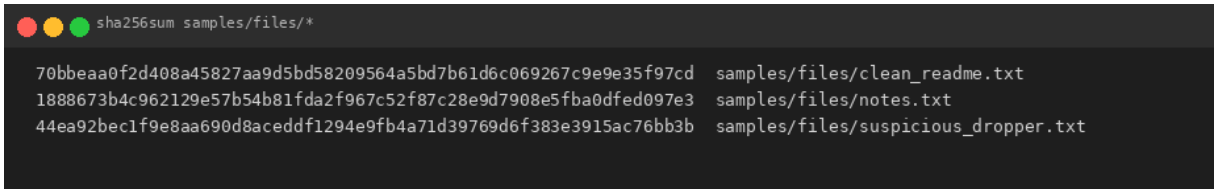

Figure 2: SHA-256 verification and cargo run output

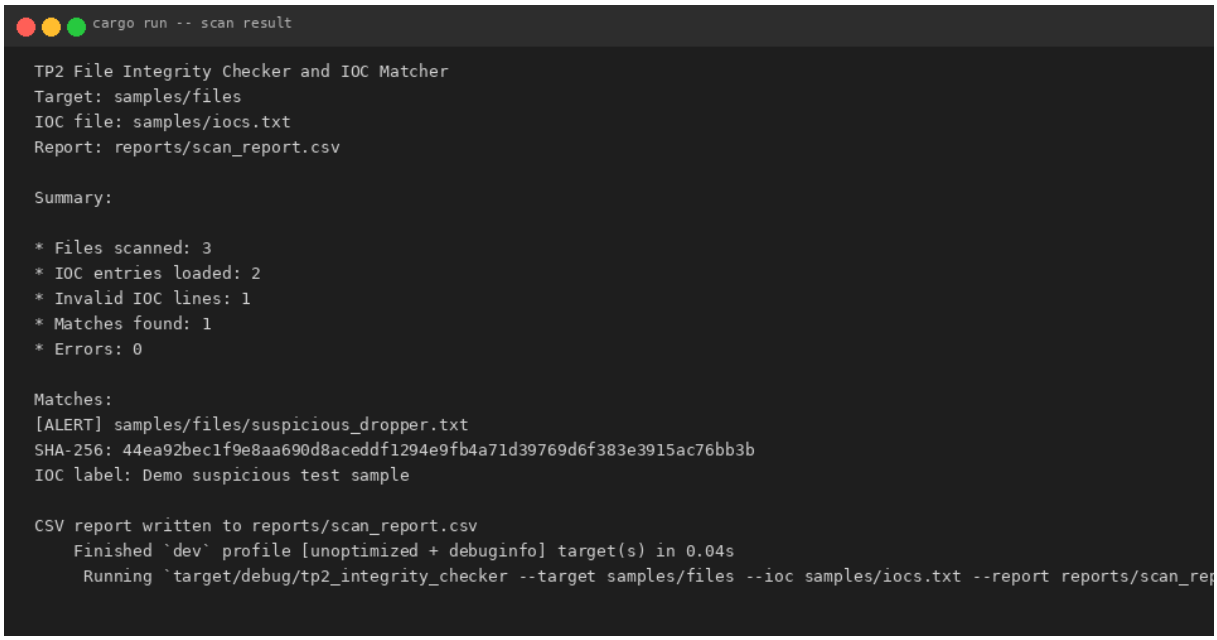

Figure 3: Scan result with match detection

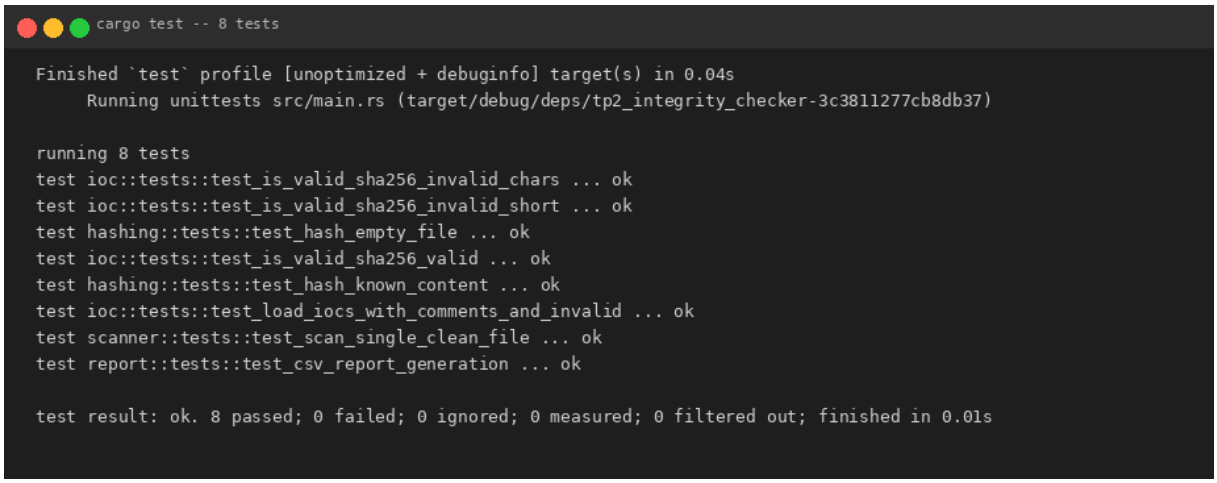

Figure 4: cargo test - all 8 tests pass

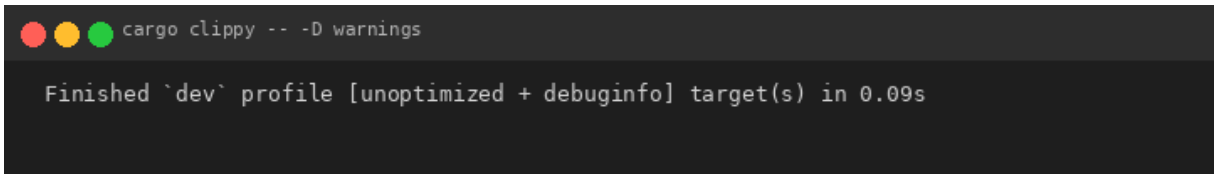

Figure 5: cargo clippy - no warnings

Generated CSV report content:

```
path,sha256,status,label
samples/files/clean_readme.txt,70bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd,CLEAN,
samples/files/suspicious_dropper.txt,44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,MATCH,Dem
o suspicious test sample
samples/files/notes.txt,1888673b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3,CLEAN,
```

5. Discussion

5.1 Invalid IOC Handling

The IOC file parser gracefully handles invalid lines: lines that are not 64-character hexadecimal strings are counted as invalid and skipped. The program continues execution and reports the number of invalid lines in the summary. This ensures that a malformed IOC file does not crash the scanner.

5.2 Missing File Handling

If the target file or directory does not exist, the program prints a clear error message (e.g., 'Error: target 'foo' does not exist') and exits with a non-zero status code. The IOC file is also validated before scanning begins.

5.3 Why SHA-256 Over MD5 or SHA-1

SHA-256 is preferred over MD5 and SHA-1 for security applications because both MD5 and SHA-1 are cryptographically broken. MD5 is vulnerable to collision attacks (an attacker can create two files with the same MD5 hash), and SHA-1 has demonstrated practical collision attacks (SHAttered, 2017). SHA-256, part of the SHA-2 family, remains collision-resistant and is the current NIST standard for cryptographic hashing.

5.4 Possible Improvements

- Recursive directory scanning to handle nested directories
- JSON output format for easier integration with other tools
- --only-matches flag to show only suspicious files
- Known-good manifest mode (allowlist instead of denylist)
- Parallel scanning using Rust threads for better performance
- Integration tests with the samples/ directory
- Cargo audit integration for dependency vulnerability scanning

6. Conclusion

This TP successfully implemented a File Integrity Checker and IOC Matcher in Rust. The tool reads files safely, calculates SHA-256 hashes, parses IOC files with robust error handling, and generates reproducible CSV reports.

The following Rust concepts were practiced:
- Project structure with multiple modules (hashing, ioc, scanner, report)
- External crate dependencies (sha2, csv)
- Result and Option for error handling without panics
- Struct definitions with derived traits (Debug, Clone, PartialEq, Eq)
- Enum types for scan status (Clean, Match, Error)
- File I/O with buffered reading
- Directory traversal with std::fs::read_dir
- Unit testing with 8 test cases
- Code formatting (cargo fmt) and linting (cargo clippy)

All mandatory requirements are satisfied: the tool runs in the Docker Compose environment, uses SHA-256, avoids unsafe code and uncontrolled unwrap(), handles invalid IOC lines gracefully, generates CSV reports, and passes cargo fmt, cargo clippy, and cargo test.